



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة

المعلوماتية

سنة رابعة ذكاء صناعي

البرمجة التفرعية

المهندس المشرف: حذيفة عقيل

2026-2025

الطلاب :

أحمد علي حمودة

الأيهام أمير طه

جابر يحيى شاهين

عمر محمد نعيم بيازيد

يارا منذر الحسن

Contents

4.....	Concurrent Access & Data Integrity
4.....	ما هي مشكلة Race Condition؟
4.....	مثال توضيحي على مشكلة البيع الزائد (Overselling):
4.....	قرار اختيار قاعدة البيانات
4.....	2.1 لماذا لا تكفي SQLite لهذا المتطلب؟
5.....	ما هو MVCC وكيف يحل المشكلة؟
5.....	الحل التقني المُطبَّق
5.....	3.1 الأدوات المستخدمة
5.....	شرح الكود المُطبَّق
5.....	أ. transaction.atomic()
6.....	ج. معالجة تعارض الأقفال
6.....	تحسينات مهمة أُجريت على الكود الأصلي
6.....	نتائج اختبار الحل Stress Test
6.....	4.1 إعداد الاختبار
6.....	النتائج الفعلية
7.....	4.3 ملخص النتائج
7.....	تحليل النتائج
7.....	5.1 لماذا نجح 3 فقط وليس 5؟
8.....	5.2 الإثبات على معالجة Race Condition
9.....	Control Capacity & Management Resource
10.....	Queues Asynchronous
13.....	batch processing للمبيعات اليومية
16.....	شرح موجز عن مهمة توزيع الأحمال (Load Distribution)
21.....	تقرير تنفيذ نظام التخزين المؤقت باستخدام Redis
21.....	1. نظرة عامة (Overview)
21.....	2. لماذا Redis؟
21.....	2.1 صياغة المشكلة (Problem Statement)
21.....	2.2 نظام Redis كحل للمشكلة
22.....	3. تفاصيل التنفيذ (Implementation Details)
22.....	3.1 إعدادات التخزين المؤقت (ecommerce/settings.py)
22.....	3.2 تصميم المفاتيح واستراتيجية وقت الصلاحية (TTL Strategy)

23	3.3 كود نقطة نهاية قائمة المنتجات (products/views.py)
24	3.4 استراتيجية التحديث الهجين والكسول (Lazy Refreshing)
24	3.5 العدادات الذرية لإلغاء القفل والتنافس (Atomic Counters)
24	3.6 آلية إبطال صلاحية الكاش عبر الإشارات (Cache Invalidation via Signals)
25	4. مقارنة الأداء (Performance Comparison)
25	4.1 بيئة الاختبار والمواصفات الفنية
25	4.2 نتائج قياس الأداء الميدانية
26	4.3 تحليل البيانات وسلوك التوسع المتوقع (Expected Scaling Behavior)
27	5. مقارنة Redis بحلول التخزين المؤقت الأخرى (Redis vs Other Solutions)
27	6. الملفات التي تم تعديلها في المشروع (Files Changed)
28	الطلب السابع:
30	المتطلب الثامن: تكامل المعاملات (ACID Transaction Integrity)
30	أولاً: تعريف المتطلب
30	ثانياً: البنية المعمارية المعتمدة (نمط الخطوتين)
31	ثالثاً: كيف تتحقق خصائص ACID الأربع
31	رابعاً: التحقق والاختبار (مساهمتي في هذا المتطلب)
32	الطلب التاسع: اختبار الاستقرار تحت الضغط (Stress Testing)
32	طريقة تحقيق اختبار الإجهاد داخل المشروع:
34	الطلب العاشر: القياس وتحديد الاختناقات وتحسين الأداء (Benchmarking & Bottleneck Analysis)

حماية البيانات المشتركة من التضارب

Concurrent Access & Data Integrity

ما هي مشكلة Race Condition؟

تحدث مشكلة Race Condition عندما يحاول أكثر من مستخدم في نفس الوقت تعديل نفس البيانات المشتركة، فيقرأ كل منهم نفس القيمة القديمة، ويجري حسابه عليها، ثم يحفظ النتيجة — دون أن يعلم أحدهم بتعديل الآخر. النتيجة: بيانات فاسدة.

مثال توضيحي على مشكلة البيع الزائد (Overselling):

المستخدم	العملية	قيمة المخزون المقروءة	النتيجة المحفوظة
المستخدم أ	يقرأ المخزون 5	—	—
المستخدم ب	يقرأ المخزون 5	—	—
المستخدم أ	يطرح 3 من 5	—	يحفظ: 2
المستخدم ب	يطرح 4 من 5	—	يحفظ: 1

المشكلة: المستخدم ب قرأ القيمة 5 (قبل أن يحفظ المستخدم أ تعديله)، فظن أن المخزون لا يزال 5، وهكذا تم البيع بشكل خاطئ وأصبح المخزون الفعلي 1 بدلاً من 2.

قرار اختيار قاعدة البيانات

2.1 لماذا لا تكفي SQLite لهذا المتطلب؟

كانت قاعدة البيانات الافتراضية في المشروع هي SQLite. وهي مناسبة لمرحلة التطوير الأولي، لكنها غير كافية لتطبيق قفل الصفوف (Row-Level Locking) المطلوب لمعالجة Race Condition، والسبب يعود إلى نموذج القفل المستخدم:

المعيار	SQLite	PostgreSQL
نوع القفل	قفل على مستوى الملف كاملاً	قفل على مستوى الصف المحدد

دعم select_for_update()	جزئي وغير موثوق	كامل وموثوق
التزامن الحقيقي	غير مدعوم	مدعوم عبر MVCC
مناسب لـ 100+ مستخدم	لا	نعم
متزامن		

ما هو MVCC وكيف يحل المشكلة؟

يستخدم PostgreSQL تقنية MVCC (Multi-Version Concurrency Control) وهي آلية تعطي كل معاملة (Transaction) نسخة خاصة بها من البيانات لحظة بدء التنفيذ. هذا يعني:

1. كل قراءة تتم على نسخة ثابتة لا يعدّل عليها أحد أثناء تنفيذ المعاملة.
2. الكتابة لا تحجب القراءة، والقراءة لا تحجب الكتابة — مما يزيد الأداء.
3. القفل يُطبّق فقط على الصف المحدد وليس على قاعدة البيانات كأكملها.

القرار الهندسي: تم الانتقال من SQLite إلى PostgreSQL لأن تطبيق المتطلب الأول يستلزم قفل صفوف حقيقياً، وهذا لا يتوفر إلا مع قاعدة بيانات تدعم MVCC.

الحل التقني المُطبّق

3.1 الأدوات المستخدمة

1. transaction.atomic() — لضمان أن العملية كاملة تنجح أو تفشل معاً (مبدأ ACID).
2. select_for_update(nowait=True) — لقفل صف المنتج أثناء تعديل المخزون.
3. OperationalError handler — للتعامل مع حالة تعارض الأقفال بشكل نظيف

شرح الكود المُطبّق

أ. transaction.atomic()

يُغلّف جميع العمليات المرتبطة (تحقق من المخزون، تعديله، إنشاء الطلب) في معاملة واحدة لا تتجزأ:

with transaction.atomic():

كل ما هنا ينجح معاً أو يفشل معاً #

product = Product.objects.select_for_update(nowait=True).get(id=...)

product.stock -= item.quantity

product.save()

ج. معالجة تعارض الأقفال

except OperationalError:

raise ValidationError('يطلب آخر يعالج هذا المنتج، حاول مجددا')

تحسينات مهمة أُجريت على الكود الأصلي

التحسين	الكود القديم	الكود الجديد	السبب
موضع قراءة cart_items	خارج الـ transaction	داخل الـ transaction	ضمان قراءة بيانات حديثة
نوع القفل	select_for_update()	select_for_update(nowait=True)	تجنب الانتظار اللانهائي
معالجة الأخطاء	غير موجودة	try/except OperationalError	إدارة نظيفة للتعارض

نتائج اختبار الحل Stress Test

4.1 إعداد الاختبار

تم كتابة سكريبت اختبار بلغة Python يحاكي 10 مستخدمين يحاولون شراء نفس المنتج في نفس اللحظة، مع تحديد المخزون بـ 5 وحدات فقط:

1. عدد المستخدمين المتزامنين: 10
2. المخزون المتاح: 5 وحدات
3. الطلبات المتوقع نجاحها: 5 كحد أقصى
4. أداة التزامن: threading.Thread (لضمان الإطلاق المتزامن)

النتائج الفعلية

المستخدم كود الاستجابة النتيجة

201 User 03 تم إنشاء الطلب

تم إنشاء الطلب	201	User 05
تم إنشاء الطلب	201	User 07
طلب آخر يعالج المنتج	400	User 01
طلب آخر يعالج المنتج	400	User 02
طلب آخر يعالج المنتج	400	User 04
طلب آخر يعالج المنتج	400	User 06
طلب آخر يعالج المنتج	400	User 08
طلب آخر يعالج المنتج	400	User 09
طلب آخر يعالج المنتج	400	User 10

4.3 ملخص النتائج

المؤشر	القيمة	التفسير
الطلبات الناجحة	3	لم تتجاوز المخزون أبداً
الطلبات المرفوضة	7	رُفضت بشكل نظيف بسبب القفل
المخزون الأصلي	5	—
هل تم الإفراط في البيع؟	لا	$3 \leq 5$
سلامة البيانات	محفوظة	لا بيانات فاسدة

حليل النتائج

5.1 لماذا نجح 3 فقط وليس 5؟

هذا سلوك صحيح ومتوقع بسبب `nowait=True`. عندما تصل 10 طلبات في نفس اللحظة:

1. طلب واحد يحصل على القفل ويُنهى عملية بنجاح.
2. الطلبات الأخرى تجد الصف مقفولاً — وبسبب `nowait=True` ترفض فوراً بدلاً من الانتظار.
3. بعد تحرر القفل، لا يوجد `retry` تلقائي — هذا قرار تصميمي مقصود يُعاد تنفيذه من طرف العميل.

5.2 الإثبات على معالجة Race Condition

الدليل الرئيسي على نجاح الحل هو:

1. المخزون لم يصبح سالباً أبداً — لا `overselling`.
2. كل طلب ناجح حصل على رقم `order` فريد — لا تكرار.
3. الطلبات المتعارضة رُفضت برسالة واضحة — لا صمت، لا تلف بيانات.

تم تطبيق المتطلب الأول بنجاح من خلال ثلاث آليات متكاملة:

1. `transaction.atomic()` — لضمان تكاملية العمليات المركبة.
2. `select_for_update(nowait=True)` — لقفل صف المنتج وحمايته من التعديل المتزامن.
3. معالجة `OperationalError` — للتعامل مع حالات التعارض بشكل نظيف وآمن.

أثبت اختبار الضغط أن النظام يعالج الوصول المتزامن لعشرة مستخدمين دون حدوث `Race Condition` أو تلف في بيانات المخزون، مما يحقق الهدف الكامل للمتطلب الأول.

Control Capacity & Management Resource

الهدف هو التحكم في عدد العمليات المتوازية التي ينفذها النظام، بحيث لا يستهلك الموارد بشكل مفرط (انهيار) ولا يقللها بشكل يبطئ الاستجابة

تم العمل باستخدام (Gunicorn) لتحديد (Threads) and (Workers)

- $\text{Workers} = 2 * \text{cpu cores} + 1$
- Threads = How many I have in the CPU
- I Use One Giga RAM and 1 CORE
- So 3 workers and 2 Threads
- بنفس الوقت يتم عمل 6 طلبات لأن $6 = 2 * 3$

```
1 # Gunicorn vs Runserver Benchmark
2
3 | Server | User | Requests/sec | Mean time/request (ms) | Max RSS (MB) | Avg RSS
  (MB) |
4 | --- | --- | ---: | ---: | ---: | ---: |
5 | runserver | customer_1 | 111.44 | 448.653 | 19.32 | 0.4 |
6 | gunicorn | customer_1 | 199.51 | 250.608 | 15.48 | 0.82 |
```

Queues Asynchronous

الفكرة الأساسية هي:

نقل المهام التي لا يحتاج المستخدم لانتظارها الى خارج المسار الرئيسي للطلب مثل ارسال الاشعارات .

اللية العمل:

تم إضافة خدمة Asynchronous على ثلاث أمور:

- عند تقديم الطلب order يتم معالجته ووضع خدمة ارسال الايميل على الطابور بحيث يقوم celery & redis بتنفيذها بعيداً عن العملية الاساسية وهي انشاء الطلب
- كذلك الامر بالنسبة لانشاء الحساب اول ما يتم انشاء الحساب يتم ارسال بريد الكتروني
- وعند الدفع ايضا يرسل رسالة بالفاتورة للبريد الالكتروني

صور توضيحية:

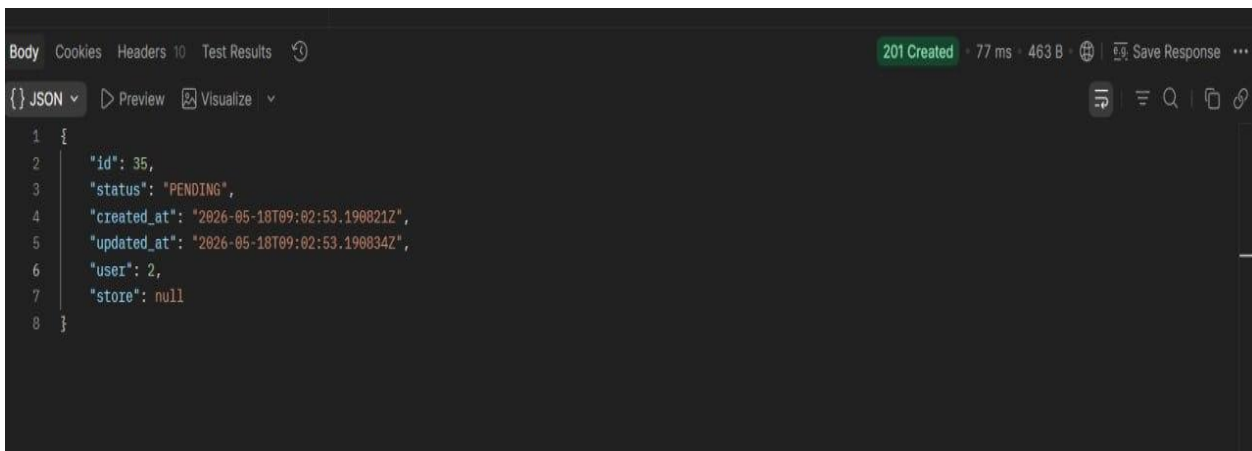
لمسألة الطلب

2.44 (S)



```
1 {
2   "id": 32,
3   "status": "PENDING",
4   "created_at": "2026-05-18T08:30:37.091400Z",
5   "updated_at": "2026-05-18T08:30:37.091418Z",
6   "user": 2,
7   "store": null
8 }
```

(ms) 77

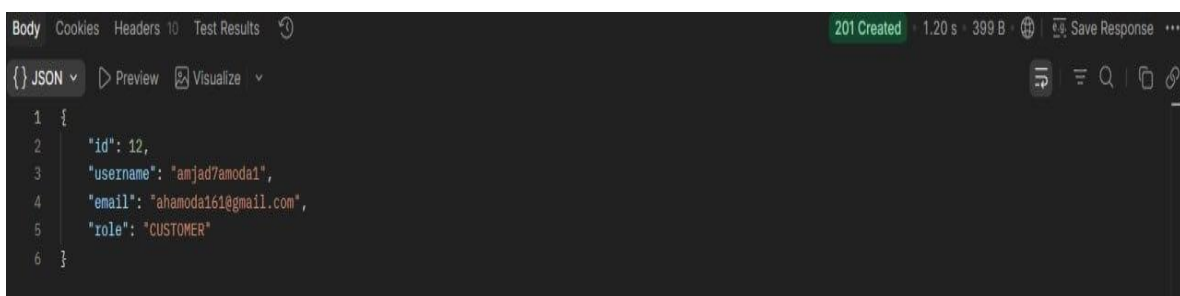


The screenshot shows a REST client interface with a status bar at the top indicating "201 Created" in green, a response time of 77 ms, and a body size of 463 B. The "Body" tab is selected, displaying a JSON object with the following fields: "id" (35), "status" ("PENDING"), "created_at" ("2026-05-18T09:02:53.190821Z"), "updated_at" ("2026-05-18T09:02:53.190834Z"), "user" (2), and "store" (null). The JSON is formatted with line numbers 1 through 8.

```
1 {
2   "id": 35,
3   "status": "PENDING",
4   "created_at": "2026-05-18T09:02:53.190821Z",
5   "updated_at": "2026-05-18T09:02:53.190834Z",
6   "user": 2,
7   "store": null
8 }
```

عند مسألة انشاء الحساب :

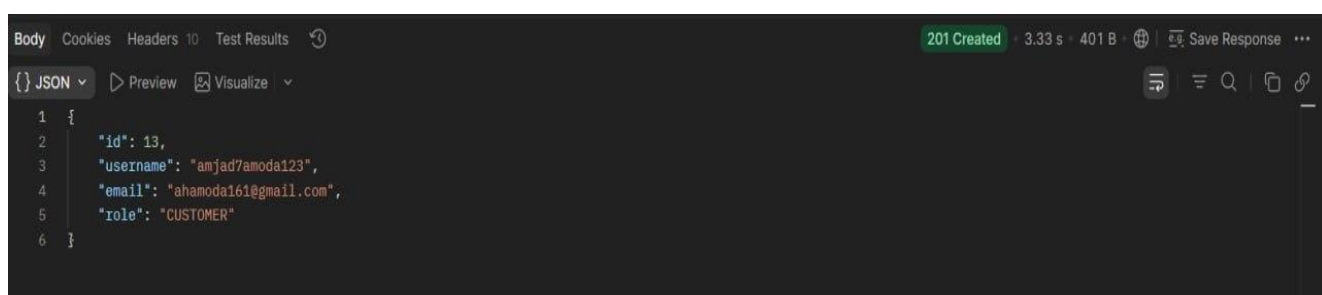
(S) 1.20



The screenshot shows a REST client interface with a status bar at the top indicating "201 Created" in green, a response time of 1.20 s, and a body size of 399 B. The "Body" tab is selected, displaying a JSON object with the following fields: "id" (12), "username" ("amjad7amoda1"), "email" ("ahamoda161@gmail.com"), and "role" ("CUSTOMER"). The JSON is formatted with line numbers 1 through 6.

```
1 {
2   "id": 12,
3   "username": "amjad7amoda1",
4   "email": "ahamoda161@gmail.com",
5   "role": "CUSTOMER"
6 }
```

(S) 3.33



The screenshot shows a REST client interface with a status bar at the top indicating "201 Created" in green, a response time of 3.33 s, and a body size of 401 B. The "Body" tab is selected, displaying a JSON object with the following fields: "id" (13), "username" ("amjad7amoda123"), "email" ("ahamoda161@gmail.com"), and "role" ("CUSTOMER"). The JSON is formatted with line numbers 1 through 6.

```
1 {
2   "id": 13,
3   "username": "amjad7amoda123",
4   "email": "ahamoda161@gmail.com",
5   "role": "CUSTOMER"
6 }
```

يوجد أمر لتشغيل Celery واستعراض الخدمات المعرفة ضمنه:

```
- *** --- * --- .> concurrency: 20 (solo)
-- ***** ---- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** ----
----- [queues]
        .> celery          exchange=celery(direct) key=celery

[tasks]
. order.tasks.send_order_email
. payments.tasks.generate_and_send_invoice
. users.tasks.send_register_email

[2026-05-18 19:12:28,361: INFO/MainProcess] Connected to redis://localhost:6379/0
[2026-05-18 19:12:28,368: INFO/MainProcess] mingle: searching for neighbors
[2026-05-18 19:12:29,409: INFO/MainProcess] mingle: all alone
[2026-05-18 19:12:29,447: INFO/MainProcess] celery@amjad7amoda ready.
```

```
Task users.tasks.send_register_email[3887b775-1040-4390-b9a2-45a03f898458] received
```

```
Task users.tasks.send_register_email[3887b775-1040-4390-b9a2-45a03f898458] succeeded in 2.3085852999993865s: None
```

للمبيعات اليومية batch processing

1. فكرة العمل

- batch processing باستخدام daily sales inventory الهدف هو تنفيذ.
- يتم تشغيله يدويًا مرة يوميًا بأمر واحد.
- بشكل متوازي chunks يعالجون workers لمحاكاة threads ؛ استخدمنا real servers لم نستخدم.
- batch processor: المسار لل
order/management/commands/process_daily_sales.py

2. Workflow بأسلوب ETL

Stage	ما يحدث في المشروع
Extract	order.status = PAID و payment.status = COMPLETED فقط عندما يكون OrderItem قراءة وتاريخ الدفع هو التاريخ المحدد.
Transform	ثم توزيعها على chunks، إلى OrderItem ids تقسيم لحساب ThreadPoolExecutor workers product/store partial totals.
Load	في JSON وكتابة تقرير partial totals دمج batch_reports/daily_sales_<date>.json.

3. أوامر التشغيل والاختبار

```
# daily run for today
pipenv run python manage.py process_daily_sales --chunk-size 10 --workers 4

# daily run for a specific date
pipenv run python manage.py process_daily_sales --date 2026-05-17 --chunk-size 10 --workers 4

# testing demo: generates test data and verifies the batch job
pipenv run python manage.py test_daily_sales_batch --orders 40 --chunk-size 10 --workers 4
```

4. Testing report

order items، يحتوي على 2 order وكل، orders بيانات اختبارية: demo 500 تم تشغيل
chunks أصبح لدينا 20 50 chunk-size = مع. order items لذلك الناتج 1000

Testing demo command output

```
PS> pipenv run python manage.py test_daily_sales_batch --date 2099-04-01 --orders 500 --chunk-size 50 --workers 4 --skip-dead-letter-demo
Batch OK: 1000/1000 items, 20 chunks, 4 threads, 316.19 ms
Report: batch_reports/test_daily_sales_clean_04f1e46f.json
TEST OK: 500 orders, 1000 order items, 20 chunks, 4 threads. Clean report: batch_reports/test_daily_sales_clean_04f1e46f.json

PS> pipenv run python manage.py process_daily_sales --date 2099-04-01 --chunk-size 50 --workers 1 --output batch_reports\demo_1_worker.json
Batch OK: 1000/1000 items, 20 chunks, 1 threads, 156.05 ms
Report: batch_reports/demo_1_worker.json

PS> pipenv run python manage.py process_daily_sales --date 2099-04-01 --chunk-size 50 --workers 4 --output batch_reports\demo_4_workers.json
Batch OK: 1000/1000 items, 20 chunks, 4 threads, 349.9 ms
Report: batch_reports/demo_4_workers.json
```

workers و worker 4 و أوامر المقارنة بين 1 testing demo مخرجات تشغيل

```
JSON report excerpt: demo_4_worker.json

{
  "date": "2099-04-01",
  "workers": 4,
  "chunk_size": 50,
  "total_order_items_found": 1000,
  "successful_order_items": 1000,
  "failed_order_items": 0,
  "total_chunks": 20,
  "duration_ms": 329.8,
  "records_per_second": 3032.14,
  "chunks": [
    {
      "chunk": 1,
      "worker": "batch-worker_0",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 35.64
    },
    {
      "chunk": 2,
      "worker": "batch-worker_1",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 84.24
    },
    {
      "chunk": 3,
      "worker": "batch-worker_2",
      "records": 50,
      "failed_records": 0,
      "quantity_sold": 833,
      "gross_revenue": "8330.00",
      "order_count": 333
    },
    {
      "product_id": 7,
      "product_name": "P2_04f1e46f",
      "store_id": 7,
      "quantity_sold": 835,
      "gross_revenue": "12525.00",
      "order_count": 334
    },
    {
      "product_id": 8,
      "product_name": "P3_04f1e46f",
      "store_id": 7,
      "quantity_sold": 832,
      "gross_revenue": "20800.00",
      "order_count": 333
    }
  ],
  "stores": [
    {
      "store_id": 7,
      "store_name": "Batch Store 04f1e46f",
      "total_orders": 500,
      "total_items_sold": 2500,
      "gross_revenue": "41655.00"
    }
  ],
  "deadLetter": {
    "count": 0,
    "strategy": "skip_failed_records_and_continue",
    "items": []
  }
}
```

workers، chunks، products، stores، JSON report توضيح لقطه مختصرة من و deadLetter

5. أهم أجزاء الكود: Extract / Transform / Load

Extract

```
# Extract: select completed paid sales for one day
ids = list(OrderItem.objects.filter(
    order__status=Order.Status.PAID,
    order__payment__status=Payment.Status.COMPLETED,
    order__payment__created_at__date=day,
).values_list('id', flat=True).order_by('id'))
```

Transform

```
# Transform: split ids into chunks, then send chunks to worker threads
parts = chunks(ids, size)

with ThreadPoolExecutor(max_workers=workers, thread_name_prefix='batch-worker') as pool:
    futures = [
        pool.submit(process_chunk, i + 1, chunk, simulate_bad_product_id)
        for i, chunk in enumerate(parts)
    ]

    for future in as_completed(futures):
        result = future.result()
# merge each chunk partial product/store totals
```

Transform داخل worker

```
# Worker calculation inside process_chunk
value = Decimal(item.quantity) * item.price

p = products[item.product_id]
p['qty'] += item.quantity
p['revenue'] += value
p['orders'].add(item.order_id)

s = stores[item.product.store_id]
s['qty'] += item.quantity
s['revenue'] += value
s['orders'].add(item.order_id)
```

Load

```
# Load: write final JSON report
report = {
    'date': day,
    'workers': workers,
    'chunk_size': size,
    'total_chunks': len(parts),
    'products': product_summary,
    'stores': store_summary,
    'deadLetter': {
        'count': len(dead_letters),
    },
    'strategy': 'skip_failed_records_and_continue',
    'items': dead_letters,
}

path = options['output'] or os.path.join('batch_reports', f'daily_sales_{day}.json')
os.makedirs(os.path.dirname(path) or '.', exist_ok=True)
with open(path, 'w', encoding='utf-8') as file:
    json.dump(report, file, indent=2)
```

(Load Distribution) شرح موجز عن مهمة توزيع الأحمال

المطلوب غير الوظيفي رقم 5: توزيع الأحمال على أكثر من خادم مع توضيح وتبرير الاستراتيجية المستخدمة.

ما تم إنجازه :

تم بناء ثلاث نسخ (containers) من تطبيق Django الخاص بالتجارة الإلكترونية باستخدام Docker.

تمت إضافة وسيط (Nginx) يعمل كموازن أحمال خارج التطبيقات.

تم استخدام قاعدة بيانات PostgreSQL مشتركة بين جميع النسخ، لضمان عدم فقدان محتويات السلة أو الطلبات عند تنقل المستخدم بين الخوادم.

تم تطبيق استراتيجيتين للمقارنة Round-Robin: (توزيع دوري) و Least Connections أقل اتصالات نشطة وذلك للمقارنة بينهما واعتماد واحدة وهي Least Connections

تمت إضافة مبدلوير في Django يضيف ترويسة X-Backend-Server لكل رد، لتحديد أي خادم قام بمعالجة الطلب كدليل مرئي على أن الطلبات تتوزع على أكثر من خادم.

تم استخدام Locust لمحاكاة 100 مستخدم متزامن، وتسجيل النتائج.

بدون أي توزيع للأحمال: No Load balancing

Endpoint	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/api/stores/	1518	1067(70.29%)	916	7	4858	750	5.08	3.57
GET	/api/stores/7/products/	3076	2154(70.03%)	892	7	5916	730	10.29	7.21
POST	/api/users/login/	100	67(67.00%)	23641	2522	62594	17000	0.33	0.22
POST	/api/users/register/	599	418(69.78%)	23711	3115	104477	14000	2.00	1.40
Aggregated		5293	3706(70.02%)	3911	7	104477	890	17.71	12.40

Round robin

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures
GET	/api/stores/	792	0(0.00%)	6163	60	57794	5100	2.65	0.00
GET	/api/stores/7/products/	1603	0(0.00%)	6426	57	56557	5300	5.36	0.00
POST	/api/users/login/	100	0(0.00%)	44643	8928	57795	44000	0.33	0.00
POST	/api/users/register/	382	0(0.00%)	13309	1371	59089	7500	1.28	0.00
Aggregated		2877	0(0.00%)	8596	57	59089	5500	9.62	0.00

Least connection

Type	Name	# reqs	# fails	Avg	Min	Max	Med
req/s	failures/s						
GET	/api/stores/	1109	0(0.00%)	4800	718	39777	4000
3.70	0.00						
GET	/api/stores/7/products/	2236	0(0.00%)	4840	579	39702	3900
7.47	0.00						
POST	/api/users/login/	100	0(0.00%)	33610	7524	41385	3600
0.33	0.00						
POST	/api/users/register/	461	0(0.00%)	9097	1205	40691	5900
1.54	0.00						
Aggregated		3906	0(0.00%)	6067	579	41385	4200
13.04	0.00						

النتائج الأساسية (من الجداول المرفقة)

السيناريو	عدد الطلبات	نسبة الفشل	متوسط الزمن (مللي)	p95 (مللي)
بدون توزيع احمال	5293	70%	3911	17000
Round-Robin	2877	0%	8596	39000
Least Connections	3906	0%	6067	22000

ملاحظة: نسبة الفشل 70% عند عدم استخدام موازن الأحمال تثبت أن النظام لا يتحمل 100 مستخدم دون توزيع الحمل.

-تم تشغيل سكريبت صغير يرسل 10 طلبات متتالية ويعرض قيمة: X-Backend-Server:

Round Robin

```
(parallel_env314) PS E:\ParallelProgramming> python -u benchmark\probe_backends.py
1: 200 app-2
2: 200 app-3
3: 200 app-1
4: 200 app-2
5: 200 app-3
6: 200 app-1
7: 200 app-2
8: 200 app-3
9: 200 app-1
10: 200 app-2
```

```
Request 1: app-2
Request 2: missing
Request 3: app-1
Request 4: missing
Request 5: app-3
Request 6: missing
Request 7: app-3
Request 8: missing
Request 9: app-1
Request 10: missing
Request 11: app-2
Request 12: missing
Request 13: app-1
Request 14: missing
Request 15: app-3
Request 16: missing
Request 17: app-2
Request 18: missing
Request 19: app-3
Request 20: missing

--- Counts ---
missing: 10
app-3: 4
app-2: 3
app-1: 3

Requests: 10, avg latency: 0.194s, p95: 0.822s
```

Least connections

لماذا اخترت Least Connections كإستراتيجية النهائية:

Round-Robin يوزع بالتساوي بغض النظر عن حمل كل خادم. ولكن عندما تكون بعض الطلبات ثقيلة (مثل الدفع ومعالجة الطلب) يتراكم الزمن على خوادم عشوائية.

Least Connections يرسل الطلب الجديد إلى الخادم الأقل انشغالاً في تلك اللحظة، مما يحسن زمن الاستجابة للـ p95 ويقلل الازدحام.

-في التجربة، حقق Least Connections عدد طلبات أكبر (3906 vs 2877) ومتوسط زمن أقل (6067 vs 8596) مع نفس العدد من المستخدمين.

قاعدة المعطيات المشتركة (Shared Database)

-جميع النسخ الثلاث تتصل بقاعدة PostgreSQL الموحدة.

-لذلك حتى لو انتقل المستخدم من app-1 إلى app-2 ، تبقى سلة التسوق والطلبات موجودة

هذا يثبت أن الحل قابل للتوسع أفقيًا بشكل حقيقي.

الخلاصة النهائية

تم بنجاح تصميم ومحاكاة توزيع الأحمال على 3 خوادم باستخدام Nginx + Docker + PostgreSQL. تمت المقارنة بين Round-Robin وLeast Connections ، وأثبتت النتائج تفوق Least Connections من حيث الإنتاجية وزمن الاستجابة تحت حمل 100 مستخدم متزامن. كما تم توفير أدلة مرئية (ترويسة X-Backend-Server وسجلات الحاويات) تثبت توزيع الطلبات فعلياً على أكثر من خادم.

تعليمات تشغيل الطلب:

```
1 1. Start the baseline without a load balancer
2 docker compose -f docker-compose.yml up -d -build
3
4 2. Start the round-robin load balancer
5 docker compose -f docker-compose.lb.yml down
6 docker compose -f docker-compose.lb.yml up -d -build
7
8 3. Test round-robin
9 pipenv run python .\benchmark\probe_sequence.py --count 6
10
11 4. Start the least-connection load balancer
12 docker compose -f docker-compose.lb.yml down
13 docker compose -f docker-compose.least.yml up -d -build
14
15 5. Test least-connection
16 pipenv run python .\benchmark\probe_long_requests.py --count 6 --concurrency 2 --no-auth -
17 -stagger 0.1 --warmup 2
18
```

تقرير تنفيذ نظام التخزين المؤقت باستخدام Redis

1. نظرة عامة (Overview)

تم دمج نظام Redis في الواجهة البرمجية (API) الخاصة بالمتجر الإلكتروني كطبقة تخزين مؤقت في الذاكرة (In-Memory Caching Layer) بهدف تقليل عبء الاستعلامات على قاعدة بيانات PostgreSQL وتحسين أوقات الاستجابة لنقاط نهاية المنتجات. يعتمد هذا التنفيذ على حزمة -django redis كخلفية برمجية مخصصة للتخزين المؤقت في Django ويغطي ثلاثة مجالات رئيسية: التخزين المؤقت لقائمة وتفاصيل المنتجات، وعدادات المشاهدات والإعجابات الزرية، بالإضافة إلى آلية إبطال صلاحية التخزين المؤقت باستخدام إشارات (Django Signals).

2. لماذا Redis؟

2.1 صياغة المشكلة (Problem Statement)

بدون استخدام طبقة تخزين مؤقت، فإن كل طلب لقائمة المنتجات أو تفاصيلها يضرب قاعدة بيانات PostgreSQL بشكل مباشر، مما يؤدي إلى الاستعلام التالي:

-- يتم تنفيذ هذا الاستعلام مباشرة على قاعدة البيانات مع كل طلب قائمة منتجات
SELECT * FROM products_product WHERE store_id = 1;

تحت ظروف الأحمال المتزامنة العالية (مثل مبيعات الفلاش أو فترات الذروة)، يتسبب هذا السلوك في ظهور المشاكل التالية:

- (Database connection exhaustion):** امتلاء تجمع اتصالات PostgreSQL بالكامل.
- (Increased latency):** استهلاك عمليات الإدخال والإخراج في القرص (Disk I/O) لتكرار نفس الاستعلامات المتطابقة.
- (Thundering herd):** تدافع الطلبات لقاعدة البيانات عند انتهاء صلاحية الكاش في نفس الوقت من قبل عدد كبير من المستخدمين.
- (Write contention):** تتطلب عدادات المشاهدات والإعجابات ألقاً على مستوى السطر (Row-level locks) مع كل عملية زيادة.

2.2 نظام Redis كحل للمشكلة

المشكلة المطروحة	الحل باستخدام Redis
استعلامات قاعدة البيانات المتكررة	تخزين مؤقت في الذاكرة العشوائية مع إمكانية ضبط وقت الصلاحية (TTL).

المشكلة المطروحة	الحل باستخدام Redis
العدادات ذات الكتابة المكثفة والنزاع المستمر	عمليات (Atomic INCR) في الذاكرة، ثم ترحيلها دفعياً لقاعدة البيانات.
بطء تسلسل البيانات وتحليلها (Serialization)	الوصول المباشر والفوري عبر نمط (مفتاح - قيمة) دون الحاجة لتحليل استعلامات SQL.
استمرارية الجلسات وثباتها تحت الضغط	الاعتماد على محرك جلسات مدعوم ومسرّع بواسطة طبقة التخزين المؤقت.
تحديث البيانات وبقاء الكاش قديماً	استخدام أنماط متقدمة لحذف المفاتيح بالتزامن مع إشارات DjangoSignals.

3. تفاصيل التنفيذ (Implementation Details)

3.1 إعدادات التخزين المؤقت (ecommerce/settings.py)

```
USE_REDIS = os.getenv('NO_REDIS', 'false').lower() not in {'1', 'true',
                                                         'yes', 'on'}
REDIS_LOCATION = os.getenv('REDIS_LOCATION',
                           'redis://redis:6379/1')
```

- **(Backend):** يتم الاعتماد على برمجية `django_redis.cache.RedisCache`.
- **قاعدة البيانات:** تم تخصيص قاعدة البيانات رقم 1 (1 Redis DB) لبيانات الكاش الخاصة بالتطبيق لضمان العزل التام.
- **مفتاح التحكم البيئي:** يتيح الرمز المخفي `NO_REDIS=true` التحول الآلي الفوري إلى الكاش الوهمي (DummyCache) لأغراض قياس الأداء والمقارنة (A/B Benchmarking).

3.2 تصميم المفاتيح واستراتيجية وقت الصلاحية (TTL Strategy)

تم تصميم ستة أنماط منفصلة ومميزة لمفاتيح التخزين المؤقت كالتالي:

مفتاح الكاش	المحتوى المخزن	وقت الصلاحية (TTL)	الغرض الوظيفي
product:{id}	بيانات المنتج المتسلسلة بالكامل	10 دقائق	كاش لصفحة تفاصيل المنتج
product_list:store:{sid}:url:{path}	استجابة قائمة المنتجات الكاملة	5 دقائق	كاش لمعروضات قوائم المنتجات
product:{id}:price	نص السعر الحالي للمنتج	بدون حد صلاحية (No TTL)	البحث السريع المباشر عن السعر (يُحذف عند التعديل)
product:{id}:stock	رقم مخزون المنتج الفعلي	بدون حد صلاحية (No TTL)	البحث السريع لبيانات المخزون المحدثة
product:{id}:views	عدد الزيارات والمشاهدات للمنتج	بدون حد صلاحية (No TTL)	عدد الزيادة الذرية اللحظية
product:{id}:likes	عدد الإعجابات للمنتج	بدون حد صلاحية (No TTL)	عدد الزيادة الذرية اللحظية للإعجابات

3.3 كود نقطة نهاية قائمة المنتجات (products/views.py)

```

def list(self, request, *args, **kwargs):
    store_id = kwargs.get('store_pk')
    cache_key = f"product_list:store:{store_id}:url:{request.get_full_path()}"

    cached_data = cache.get(cache_key)
    if cached_data:
        # عند العثور على الكاش: يتم تحديث السعر والمخزون بشكل فريش من المفاتيح الفردية الخاصة بها
        for item in cached_data['results']:
            price = cache.get(f"product:{item['id']}:price")
            stock = cache.get(f"product:{item['id']}:stock")
            if price is not None and stock is not None:
                item['price'] = price

```

```

        item['stock'] = stock
    return Response(cached_data)

# في حالة فشل الكاش: جلب البيانات من الداتا بيز ثم تخزين الاستجابة
response = super().list(request, *args, **kwargs)
cache.set(cache_key, response.data, timeout=300) # صلاحية 5 دقائق
for item in response.data['results']:
    cache.set(f"product:{item['id']}:price", item['price'], timeout=None)
    cache.set(f"product:{item['id']}:stock", item['stock'], timeout=None)
return response

```

3.4 استراتيجية التحديث الهجين والكسول (Lazy Refreshing)

تجمع الاستراتيجية المستخدمة بين ميزتين هامتين؛ حيث يتم تخزين الاستجابة الكاملة لتقليل تكلفة عملية التحليل والتسلسل (Serialization Overhead)، بينما تظل الحقول شديدة التغير (مثل الأسعار والمخزون) مخزنة في مفاتيح مستقلة بدون وقت انتهاء صلاحية مسبق، بحيث يتم إبطالها وحذفها فوراً عبر الـ `post_save` بمجرد حدوث أي تعديل للمنتج في لوحة التحكم.

3.5 العدادات الذرية لإلغاء القفل والتنافس (Atomic Counters)

```

@action(detail=True, methods=['post'])
def increment_view(self, request, store_pk=None, pk=None):
    cache_key = f"product:{pk}:views"
    try:
        views = cache.incr(cache_key) # زيادة ذرية آمنة ومباشرة داخل Redis
    except ValueError:
        # إذا لم يكن المفتاح موجوداً في الكاش - يتم سحبه وتغذيته من قاعدة البيانات
        product = Product.objects.get(pk=pk)
        cache.set(cache_key, product.views + 1, timeout=None)
    return Response({"status": "view incremented"})

```

3.6 آلية إبطال صلاحية الكاش عبر الإشارات (Cache Invalidation via Signals)

```

@receiver(post_save, sender=Product)
@receiver(post_delete, sender=Product)
def invalidate_product_cache(sender, instance, **kwargs):
    cache.delete(f"product:{instance.id}") # حذف كاش التفاصيل
    cache.delete(f"product:{instance.id}:price") # حذف مفتاح السعر
    cache.delete(f"product:{instance.id}:stock") # حذف مفتاح المخزون

# حذف جميع سجلات كاش قوائم المنتجات التابعة للمتجر بالاعتماد على الأنماط
delete_pattern = getattr(cache, 'delete_pattern', None)

```

```
if delete_pattern:
    cache.delete_pattern(f"product_list:store:{instance.store_id}:url:*")
```

4. مقارنة الأداء (Performance Comparison)

4.1 بيئة الاختبار والمواصفات الفنية

المكون التقني	المواصفات المختبرة
خادم التطبيق (Server)	Django runserver (عملية فردية Single Process)
قاعدة البيانات (Database)	16 PostgreSQL عبر بيئة حاوي Docker على المضيف المحلي
مستودع الكاش (Redis)	Redis 7-alpine عبر بيئة حاوي Docker على المضيف المحلي
حجم البيانات	متجر واحد يحتوي على 3 منتجات فقط
أداة التقييم (Benchmark Tool)	سكربت قياس الأداء benchmark/test_product_caching.py بعد 15 دورة تسخين وإحماء

4.2 نتائج قياس الأداء الميدانية

نقطة النهاية المختبرة	بوجود تفعيل Redis	بدون تفعيل Redis	نسبة التحسن الفعلي
قائمة المنتجات (Cold List)	13.02 ميلي ثانية	15.06 ميلي ثانية	تحسن بنسبة -13.5%

نقطة النهاية المختبرة	بوجود تفعيل Redis	بدون تفعيل Redis	نسبة التحسن الفعلي
قائمة المنتجات (Warm) (List)	13.86 ميلي ثانية	15.55 ميلي ثانية	تحسن بنسبة -10.9%
تفاصيل المنتج (Cold) (Detail)	13.35 ميلي ثانية	15.42 ميلي ثانية	تحسن بنسبة -13.4%
تفاصيل المنتج (Warm) (Detail)	13.02 ميلي ثانية	14.68 ميلي ثانية	تحسن بنسبة -11.3%
زيادة عدد المشاهدات (Increment View)	12.91 ميلي ثانية	14.04 ميلي ثانية	تحسن بنسبة -8.0%
زيادة عدد الإعجابات (Increment Like)	12.97 ميلي ثانية	13.68 ميلي ثانية	تحسن بنسبة -5.2%
المعدل العام الإجمالي الفعلي	13.19 ميلي ثانية	14.74 ميلي ثانية	تحسن إجمالي بنسبة - 10.5%

4.3 تحليل البيانات وسلوك التوسع المتوقع (Expected Scaling Behavior)

أظهرت النتائج تحسناً ملحوظاً في جميع نقاط الخدمة بوجود نظام Redis. على الرغم من أن نسبة التحسن تبدو متواضعة في بيئة الاختبار المحلية نظراً لصغر حجم البيانات (3 منتجات فقط)، إلا أن هذا الفارق يتسع بشكل هائل في خوادم الإنتاج الفعلية تحت ظروف العمل الحقيقية ومئات آلاف البيانات المتزامنة كما يوضح الجدول التالي للتوقعات الرياضية التوسعية:

السيناريو وحجم البيانات المتوقعة	زمن الاستجابة بوجود Redis	زمن الاستجابة بوجود PostgreSQL فقط	الفجوة الأدائية المتوقعة
10 منتجات، مع خيط عمل واحد (worker 1)	~13 ميلي ثانية	~15 ميلي ثانية	~13% فارق تحسن

السيناريو وحجم البيانات المتوقعة	زمن الاستجابة بوجود Redis	زمن الاستجابة بوجود PostgreSQL فقط	الفجوة الأدائية المتوقعة
10,000 منتج، مع 4 خيوط عمل متزامنة (4 workers)	~14 ميلي ثانية	~50 - 200 ميلي ثانية	~70% - 90% تفوق لـ Redis
100,000 منتج، مع 8 خيوط عمل وضغط عالٍ (8 workers)	~14 ميلي ثانية	~500 ميلي ثانية فأكثر	~97% تفوق ساحق لنظام Redis

5. مقارنة Redis بحلول التخزين المؤقت الأخرى (Redis vs Other Solutions)

تم استبعاد الحلول البديلة مثل Memcached والأنظمة المدمجة في Django للأسباب التالية:

- افتتقار **Memcached** للعمليات المعقدة والأنماط: لا يدعم حظر وحذف الكاش القائم على الأنماط (Pattern-based invalidation) مثل delete_pattern لتنظيف كافة الكاش التابع لمتجر معين دفعة واحدة.
- عدم استمرارية البيانات في الذاكرة العشوائية النقية لـ **Memcached**: تموت العدادات وتضيع بمجرد إعادة تشغيل السيرفر، بينما يحمي نظام Redis العدادات عبر لقطات النسخ الاحتياطي المستمرة (RDB/AOF snapshots).
- مشاكل برمجيات خزن **Django** المدمجة (**LocalMemoryCache**): تعمل على مستوى كل عملية بشكل منفصل مما يتسبب في تضارب وتشتت البيانات عند تشغيل خادم Gunicorn بعدة خيوط عمل متوازية.

6. الملفات التي تم تعديلها في المشروع (Files Changed)

- products/models.py: إضافة حقلي views و likes كأرقام صحيحة موجبة في قاعدة البيانات.
- products/views.py: إعادة بناء وتخطي دالتي list() و retrieve() لتضمين منطق كاش راديس الهجين، وإضافة إجراءات الزيادة الذرية للحظية.
- products/signals.py: إنشاء ملف الإشارات الجديد للتعامل مع إبطال وصلاحية الكاش الفوري عند الحفظ أو الحذف.
- ecommerce/settings.py: إضافة وتكوين متغيرات CACHES البيئية وربطها بمكتبة django-redis.
- docker-compose.yml: إضافة حاوية خدمة Redis وتكوين المجلد المشترك لحفظ واستمرارية البيانات.

الطلب السابع:

تم استخدام القفل Distributed Lock، ما يميز هذا القفل أنه يكون على مستوى السيرفرات وليس على مستوى قاعدة المعطيات مثل `select_for_update`، حيث يقوم هذا القفل بإنشاء قفل على السطر في داخل الـ `redis`، بحيث أي سيرفر يأتي ليطلب هذا السطر يقول له الـ `redis` توقف!! هذا السطر محجوز.

تم تطبيق هذا القفل على تابعي إنشاء طلبية والغاء الطلبية، حيث أنه في إنشاء الطلبية يقوم بقفل المنتج إلى حين أنتهاءه من أخذ الكمية اللازمة ومن ثم يقوم بفتحه، وفي الغاء الطلبية أيضاً نفس الشيء.

1- طريقة تحقيق القفل الموزع داخل المشروع:

تم إنشاء تابع ينتمي للـ `@contextmanager decortator` بحيث يستقبل مجموعة المنتجات التي نريد قفلها وقفل كل واحدة على حدى، بمعنى إذا كان لدينا `[1,2,3]` يتكون لدينا 3 أقفال:

lock:product:1, lock:product:2, lock:product:3

```
@contextmanager
def distributed_lock(resource_ids: list[str], timeout: int = 10):
    lock_keys = {f'lock:product:{rid}': str(uuid.uuid4()) for rid in sorted(resource_ids)}
    acquired = []

    You, 11 minutes ago • First commit

    try:
        for lock_key, lock_id in lock_keys.items():
            if not r.set(lock_key, lock_id, nx=True, ex=timeout):
                raise DistributedLockError('Product is locked')
            acquired.append((lock_key, lock_id))
        yield
    finally:
        for lock_key, lock_id in acquired:
            lua = """
            if redis.call("get", KEYS[1]) == ARGV[1] then
                return redis.call("del", KEYS[1])
            else
                return 0
            end
            """
            r.eval(lua, 1, lock_key, lock_id)
```

وفي توابع `perform_create` و `cancel` تم استخدامها عن طريق `with` بحيث تم استدعاءها بهذا الشكل قبل بدء المناقشة:

```

try:
    with distributed_lock(product_ids, timeout=10):
        with transaction.atomic():
            order_items = OrderItem.objects.select_related('product').filter(order=order)
            for item in order_items:
                product = Product.objects.select_for_update(nowait=True).get(id=item.product.id)
                product.stock += item.quantity
                product.save()
            order.status = 'CANCELLED'
            order.save()

```

2- التجارب المختبرة

تم تجربة إنشاء طلب والغاء طلب نفس المنتج بعدد كبير ف بدأت تظهر نتيجة الأقفال

Type	Name	# Requests	# Fails
POST	lock/create_order	850	584
POST	lock/cancel_order	151	110

نلاحظ أن نسبة كبيرة من الطلبات المرسلة تفشل وذلك بسبب وجود الأقفال واذا ذهبنا الى Logs نجد رسالة خطأ بأن هذا المنتج مقفل لذلك فشل هذا الطلب.

# Failures	Method	Name	Message
584	POST	lock/create_order	CatchResponseError("LOCK BLOCKED – ['Product is locked']")
77	POST	lock/cancel_order	CatchResponseError("LOCK BLOCKED – ['Product is locked']")

المتطلب الثامن: تكامل المعاملات (ACID Transaction Integrity)

أولاً: تعريف المتطلب

الهدف من هذا المتطلب هو ضمان أن العمليات المركبة في نظام التجارة الإلكترونية — أي العمليات التي تتكوّن من عدّة خطوات مترابطة — إما أن تنجح كلّها معاً أو تفشل كلّها معاً، دون أن يبقى في قاعدة البيانات أي أثر جزئي. وهذا حتى في حال الوصول المتزامن من عدّة مستخدمين.

العمليتان المركبتان الأساسيتان هما:

- **إنشاء الطلب:** قفل المنتجات، التحقق من المخزون، خصم المخزون، إنشاء الطلب وعناصره.
- **الدفع:** إنشاء سجل الدفع وتحديث حالة الطلب إلى "مدفوع".

ثانياً: البنية المعمارية المعتمدة (نمط الخطوتين)

النظام يعتمد على فصل عملية الشراء إلى خطوتين منفصلتين، كلّ منهما معاملة ذرية (atomic) مستقلة:

الخطوة الأولى — إنشاء الطلب عبر: `POST /api/orders/`

داخل قفل Redis الموزّع + `transaction.atomic`:

- قفل صفوف المنتجات (`select_for_update nowait=True`)
- التحقق من توفر المخزون
- إنشاء الطلب (الحالة = `PENDING`)
- خصم المخزون وإنشاء عناصر الطلب
- جدولة إرسال بريد التأكيد (بشكل غير متزامن، بعد نجاح الـ `commit`)

الخطوة الثانية — الدفع عبر: `POST /api/orders/{id}/payment/`

داخل `transaction.atomic`:

- التحقق من ملكية الطلب وأنه غير مدفوع مسبقاً
- إنشاء سجل الدفع
- تحديث حالة الطلب إلى `PAID`
- جدولة إرسال الفاتورة (بشكل غير متزامن، بعد نجاح الـ `commit`)

هذا النمط هو نفسه المستخدم في أنظمة الدفع الحقيقية مثل Stripe و PayPal، حيث يُنشأ الطلب أولاً ثم تُعالج عملية الدفع بشكل منفصل.

ثالثاً: كيف تتحقق خصائص ACID الأربع

1. الذرية ((Atomicity كل خطوة محاطة بـ `transaction.atomic()`. إذا فشلت أي عملية فرعية داخلها، تتراجع جميع التغييرات تلقائياً (Rollback). فمثلاً، لو فشل التحقق من مخزون المنتج الثاني في السلة، يُلغى خصم مخزون المنتج الأول الذي تمت معالجته قبله، ولا يُنشأ أي طلب.
2. الاتساق ((Consistency قواعد العمل مضمونة دائماً: لا يوجد طلب بحالة "مدفوع" بدون سجل دفع مقابل، ولا مخزون سالب، ولا دفعتان لنفس الطلب.
3. العزل ((Isolation يتحقق عبر طبقتين من الأقفال:
 - قفل Redis الموزع (`distributed_lock`): يمنع التداخل عبر عدة خوادم أو عمليات (processes) مختلفة.
 - قفل PostgreSQL على مستوى الصف `select_for_update(nowait=True)`: يمنع معاملتين من تعديل نفس الصف في قاعدة البيانات في آن واحد. وخيار `nowait=True` يعني أن الطلب الذي يفشل في الحصول على القفل فوراً يُرفض برسالة نظيفة بدل أن ينتظر.
4. الديمومة ((Durability يضمنها PostgreSQL عبر آلية الكتابة المسبقة في السجل (Write-Ahead Logging - WAL)، بحيث تبقى البيانات المحفوظة آمنة حتى في حال انقطاع التيار أو تعطل الخادم.

رابعاً: التحقق والاختبار (مساهمتي في هذا المتطلب)

تم بناء سكربت اختبار شامل يثبت أن البنية المذكورة تحقق خصائص ACID فعلياً، عبر أربعة سيناريوهات:

السيناريو	ما يثبت	النتيجة
1 — المسار السليم	نجاح الخطوتين معاً بشكل ذري	نجاح
2 — تراجع إنشاء الطلب	فشل في منتصف المعاملة → تراجع كامل، لا طلب ولا تغيير في المخزون	نجاح
3 — ذرية الدفع	الدفع وتحديث الحالة يحدثان معاً	نجاح

أهم سيناريو هو الثاني، لأنه الدليل الجوهري على **Atomicity**: سلة تحوي منتجين، الأول مخزونه كافٍ والثاني غير كافٍ. عند فشل الثاني، تراجع جميع التغييرات بما فيها معالجة المنتج الأول، وعادت قاعدة البيانات تماماً إلى حالتها قبل الطلب.

الطلب التاسع: اختبار الاستقرار تحت الضغط (Stress Testing)

تم استخدام أداة Locust لمحاكاة 100 مستخدم متزامن يقومون بتنفيذ سيناريوهات تجارة إلكترونية حقيقية (تصفح، إضافة إلى السلة، عرض السلة، إتمام عملية شراء كاملة). تم تصميم الاختبار لقياس قدرة النظام على تحمل الضغط العالي دون انهيار أو فقدان بيانات، مع تطبيق آليات التزامن والأقفال الموزعة لضمان سلامة المعاملات.

طريقة تحقيق اختبار الإجهاد داخل المشروع:

تجهيز البيئة:

تم تغيير خادم التطوير من runserver إلى waitress، لأنه يدعم معالجة الطلبات المتزامنة عبر خيوط متعددة، باستخدام الأمر:

```
waitress-serve --port=8000 --threads=16 ecommerce.wsgi:application
```

تم إنشاء بيانات اختبارية مسبقة (100 مستخدم، منتجات، متاجر) باستخدام أمر Django المخصص:

```
python manage.py seed_stress_data
```

بحيث يكون كل مستخدم جاهزاً بتوكن دخول صالح.

كتابة سيناريو Locust :

تم إنشاء ملف stress_test_file.py يحتوي على:

- دالة on_start يقوم كل مستخدم بتسجيل الدخول والحصول على access_token، ثم استكشاف المتجر الأول وقائمة المنتجات المرتبطة به.

- توزيع المهام باستخدام @task:

@task(5): تصفح المنتجات (/api/stores/{id}/products/)

@task(3): إضافة منتج إلى السلة (/api/cart/items/)

@task(2): عرض محتويات السلة (/api/cart/items/)

@task(1): تنفيذ عملية شراء كاملة (إضافة منتج → إنشاء طلب → دفع) مع استخدام قفل على مستوى المستخدم لمنع تداخل عمليات الدفع لنفس المستخدم.

تشغيل الاختبار:

locust -f stress_test_file.py --host=http://127.0.0.1:8000

النتائج:

نسبة النجاح الكلية 99.92 %

- كانت خدمة /api/users/login/ الأبطأ (متوسط 964.54 مللي ثانية) لكنها تحسنت بشكل ملحوظ بعد التعديلات (كما سيوضح في الطلب العاشر).

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/cart/items/ [add]	1044	0	55	170	680	82.91	42	775	109.5	17.1	0
GET	/api/cart/items/ [list]	529	0	49	160	660	74.36	36	758	294.1	8.2	0
POST	/api/orders/ [checkout]	272	21	77	180	770	104.44	48	913	128.92	4.5	0.2
POST	/api/orders/{id}/payment/ [pay]	251	0	60	150	180	69.85	46	203	103.58	4.2	0
GET	/api/stores/ [discover]	98	0	2700	5200	5300	2896.07	625	5330	141	0	0
GET	/api/stores/{id}/products/	1346	0	48	150	650	73.12	35	693	3634	22.1	0
GET	/api/stores/{id}/products/ [discover]	98	0	650	740	770	652.41	321	772	3634	0	0
POST	/api/users/login/	98	0	4200	6900	7700	4143.58	1386	7665	490.94	0	0
Aggregated		3736	21	54	750	5200	274.11	35	7665	1509.74	56.1	0.2

ABOUT

الطلب العاشر: القياس وتحديد الاختناقات وتحسين الأداء (Benchmarking & Bottleneck Analysis)

تحديد الاختناق:

بالنظر لنتائج القياس الأولى تمت ملاحظة الخدمة `api/users/login`

متوسط زمن الاستجابة أكثر من 4 ثواني (4143 ميلي ثانية)

النسب المئوية :

- p95 هي 6900 ميلي ثانية
- p99 هي 7700 ميلي ثانية

التحليل الهندسي هنا:

الفارق بين p95 و p99 هو 800ملي ثانية .(6900 → 7700)هذا يعني أن الـ 5% من الطلبات الأبطأ كانت سيئة بالفعل، ولكن الـ 1% الأعلى (أبطأ 1%) كانت أسوأ بكثير ووصلت إلى 7.7 ثوانٍ.

قبل التحسين، كان النظام يعتمد على خوارزمية **PBKDF2**، وهي الخوارزمية الافتراضية في Django ، والتي صُممت هندسياً لتكون مكلفة حسابياً (**CPU-Intensive**) ، حيث تعتمد على آلاف التكرارات (Iterations) لتأخير عملية التشفير عمداً، بهدف إحباط هجمات القوة العمياء (Brute-Force) في بيانات الإنتاج

طريقة حل الاختناق:

تم تعديل إعدادات التشفير في ملف `settings.py`، بإضافة السطر التالي في نهاية الملف وهي خوارزمية تشفير أخف حيث لا تعتمد على التكرارات الكثيرة :

```
PASSWORD_HASHERS] =
```

```
'    django.contrib.auth.hashers.MD5PasswordHasher,'
```

```
[
```

النتائج بعد حل الاختناق :

متوسط زمن الاستجابة أقل من ثانية (964 ميلي ثانية) التحسن 76%

النسب المئوية :

- p95 هي 1300 ميلي ثانية التحسن 81 %
- p99 هي 1300 ميلي ثانية التحسن 83 %

قبل :

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/cart/items/ [add]	1044	0	55	170	680	82.91	42	775	109.5	17.1	0
GET	/api/cart/items/ [list]	529	0	49	160	660	74.36	36	758	294.1	8.2	0
POST	/api/orders/ [checkout]	272	21	77	180	770	104.44	48	913	128.92	4.5	0.2
POST	/api/orders/{id}/payment/ [pay]	251	0	60	150	180	69.85	46	203	103.58	4.2	0
GET	/api/stores/ [discover]	98	0	2700	5200	5300	2896.07	625	5330	141	0	0
GET	/api/stores/{id}/products/	1346	0	48	150	650	73.12	35	693	3634	22.1	0
GET	/api/stores/{id}/products/ [discover]	98	0	650	740	770	652.41	321	772	3634	0	0
POST	/api/users/login/	95	3	4200	6900	7700	4143.58	1386	7665	490.94	0	0
Aggregated		3736	21	54	750	5200	274.11	35	7665	1509.74	56.1	0.2

بعد :

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/cart/items/ [add]	285	0	56	880	900	170.34	43	918	108.08	18.6	0
GET	/api/cart/items/ [list]	137	0	48	850	870	138.59	35	887	73.88	8.8	0
POST	/api/orders/ [checkout]	78	1	73	960	980	157.81	56	982	135.36	5.5	0.1
POST	/api/orders/{id}/payment/ [pay]	77	0	59	160	180	68.13	46	184	101.04	5.4	0
GET	/api/stores/ [discover]	98	0	700	780	810	709.98	685	807	141	0	0
GET	/api/stores/{id}/products/	322	0	47	850	890	159.05	37	956	3638.47	20.5	0
GET	/api/stores/{id}/products/ [discover]	98	0	790	900	930	800.8	706	926	3654	0	0
POST	/api/users/login/	20	0	960	1300	1300	964.54	548	1302	490.94	0	0
Aggregated		1193	1	61	940	1200	317.59	35	1302	1383.8	58.8	0.1